# Benchmarking Methodologies and Profiling Ecosystems for Deep Learning Kernel Optimization

**Shahid Mohammad Mulla**
UC San Diego
ECE Department
smulla@ucsd.edu

**Cheng Lun Tai**
UC San Diego
ECE Department
c8tai@ucsd.edu

**Subhon Ghosh**
UC San Diego
ECE Department
sug006@ucsd.edu

## Abstract

Without rigorous benchmarking, it is easy to misattribute GPU bottlenecks and optimize the wrong component. This paper presents a systematic study of benchmarking methodologies and profiling ecosystems for GPU-accelerated deep learning, centered on three Transformer kernels: GEMM, element-wise Add, and Softmax. We construct a dual-precision Roofline Model, derive analytical FLOPs and memory traffic formulas, and profile each kernel across five problem sizes in FP32 and FP16. GEMM transitions from memory-bound to compute-bound with scale, while Add and Softmax remain persistently memory-bound, leading to fundamentally different optimization strategies for each kernel class. The entire code base for this project is available at link.

## 1 Introduction

A single Transformer forward pass invokes hundreds of CUDA kernels spanning matrix multiplications, element-wise operations, and attention normalization. Practitioners routinely apply optimizations such as mixed precision and kernel fusion, yet these are often applied without understanding why a kernel is slow. In the layered stack of OS, driver, CUDA runtime, and Python interpreter, pinpointing the bottleneck requires systematic measurement.

We address this by combining two complementary tools: the Roofline Model (1), which gives a theoretical performance ceiling for any kernel, and Nsight Systems (4)-equivalent timeline analysis at microsecond resolution. We study three kernels central to every Transformer layer (3): (i) **GEMM**, Q/K/V projections and MLP layers, (ii) **element-wise Add**, residual connections, and (iii) **Softmax**, attention score normalization. Our contributions include a measured GPU hardware characterization, a dual-precision Roofline with empirical ceilings, a rigorous benchmarking harness, and a full kernel classification with evidence-based optimization recommendations.

## 2 Related Work and Profiling Ecosystems

**Theoretical performance models.** The Roofline Model (1) maps any kernel onto a chart of operational intensity (OI, FLOP/byte) versus achieved performance (GFLOP/s), defining a memory-bandwidth roof and a flat compute ceiling. Their intersection (the ridge point) separates memory-bound from compute-bound behavior. The Cache-Aware Roofline (6) extends this to multiple memory hierarchy levels, though HBM analysis suffices for GPU work.

**Application-level profilers.** PyTorch Profiler (5) instruments Python-level operators and reports CPU/GPU time and memory per op. It is the first tool practitioners reach for, but operates at the `torch.nn` abstraction and cannot distinguish individual CUDA kernels within a single `torch.matmul` call.

**Kernel- and system-level profilers.** NVIDIA Nsight Compute (ncu) (4) provides roofline mapping, instruction throughput, cache rates, and occupancy per CUDA kernel, at the cost of one-kernel-at-a-time tracing. Nsight Systems (nsys) (4) takes a broader view, recording the full GPU timeline at nanosecond resolution with NVTX region annotations for CPU-GPU attribution. Google Perfetto (7) offers similar tracing for CPU and Android workloads.

**Common benchmarking pitfalls.** Three pitfalls are especially prevalent. (1) Async masking: CUDA launches are non-blocking; without `cuda.synchronize()` the clock stops at dispatch, not completion. (2) Cold-start bias: the first iteration triggers JIT and cuBLAS plan selection, inflating latency by orders of magnitude. (3) Clock throttling: sustained loads can reduce GPU frequency

mid-run, making early timing unrepresentative of steady state.

## 3 Hardware Characterization

### 3.1 Specification Lookup

Every GPU has two fundamental hardware limits that define the Roofline ceilings: **Peak Compute** $P_c$ (TFLOP/s) and **Peak Memory Bandwidth** $B_w$ (GB/s). These are drawn from NVIDIA datasheets. For our target GPU, an NVIDIA L4, the specification values are $P_c^{\text{FP32}} = 30.3$ TFLOP/s, $P_c^{\text{FP16}} = 242.0$ TFLOP/s (Tensor Core), and $B_w = 300.0$ GB/s, with 24 GB of GDDR6 VRAM across 58 streaming multiprocessors.

### 3.2 Empirical Measurement

Specification values are theoretical upper bounds under ideal, sustained conditions. Real-world attainment is lower due to clock throttling, memory controller overhead, and power limits. We measure empirical peaks using two micro-benchmarks.

**Bandwidth measurement.** We implement a vector TRIAD kernel, $C = A+B$, over large arrays (up to 80% of free VRAM) to saturate the HBM bus. Each invocation transfers exactly $3 \times N \times e_b$ bytes, where $N$ is the element count and $e_b$ is the element size in bytes. The empirical bandwidth is:

$$B_w^{\text{meas}} = \frac{3 \times N \times e_b}{t_{\text{ms}}/1000} \text{ (GB/s)} \tag{1}$$

**Compute measurement.** We run a large square GEMM, $C = A \times B$ for $A, B \in \mathbb{R}^{n \times n}$. The FLOPs for this operation are exactly $2n^3$ (one multiply and one add per inner-product element). The empirical compute ceiling is:

$$P_c^{\text{meas}} = \frac{2n^3}{t_{\text{ms}}/1000} \text{ (TFLOP/s)} \tag{2}$$

We sweep $n \in \{4096, 6144, 8192\}$ for FP16 and $n \in \{2048, 3072, 4096\}$ for FP32 and take the maximum across sizes. Our empirical measurements are:

- $B_w^{\text{meas}} \approx 238$ GB/s (79% of spec)
- $P_c^{\text{FP16, meas}} \approx 55.07$ TFLOP/s
- $P_c^{\text{FP32, meas (TF32)}} \approx 26.46$ TFLOP/s
- $P_c^{\text{FP32, strict}} \approx 12.03$ TFLOP/s

## 4 Roofline Model Construction

### 4.1 Model Definition

The Roofline Model captures the maximum achievable performance of any kernel as:

$$\text{Perf}(\text{OI}) = \min\big(P_c,\ \text{OI} \times B_w\big) \tag{3}$$

where OI = FLOPs/Bytes is the *operational intensity* of the kernel. For OI below the ridge point $\text{OI}^* = P_c/B_w$, performance is limited by memory bandwidth and scales linearly with OI. For OI above the ridge point, performance is limited by compute and is flat at $P_c$.

The ridge points for our GPU are:

$$\text{OI}^*_{\text{FP32}} = \frac{30.3 \times 10^3}{300} \approx 101 \text{ FLOP/Byte} \tag{4}$$

$$\text{OI}^*_{\text{FP16}} = \frac{242.0 \times 10^3}{300} \approx 807 \text{ FLOP/Byte} \tag{5}$$

This large FP16 ridge point is significant: it means almost all practical LLM kernels are memory-bound when running in FP16, except for large GEMM operations. The FP16 compute ceiling is $242/30.3 \approx 8\times$ higher than FP32, but the memory bandwidth roof is identical for both precisions because bandwidth is measured in bytes, not FLOPs.

We construct four roofline curves: FP32 and FP16, each in spec-sheet and empirically-measured form. Figure 1 shows both variants, with the memory-bound and compute-bound regions labeled.

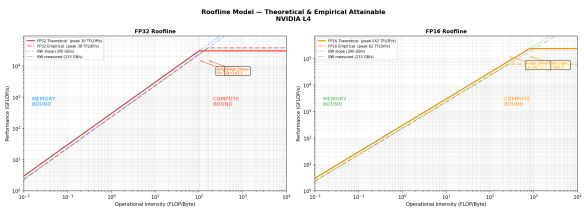


Figure 1: Roofline Model (FP32 and FP16) showing both the theoretical spec-sheet ceilings and the empirically measured attainable ceilings. Ridge points are annotated. The empirical FP32 roof sits significantly below spec due to TF32 vs strict FP32 differences.

## 5 Kernel Definitions and FLOPs Analysis

To place a kernel on the Roofline we must derive its FLOPs and memory traffic analytically. Let $B$ = batch size, $N$ = sequence length, $D$ = hidden dimension, and $e_b$ = bytes per element ($e_b = 4$ for FP32, $e_b = 2$ for FP16).

### 5.1 GEMM (Matrix Multiply)

GEMM implements $C = AW$ where $A \in \mathbb{R}^{BN\times D}$ and $W \in \mathbb{R}^{D\times D}$. Each output element requires $D$ multiplications and $D-1$ additions $\approx 2D$ FLOPs, giving a total of:

$$\text{FLOPs}_{\text{GEMM}} = 2 \times B \times N \times D^2 \tag{6}$$

Memory traffic covers reading the input matrix $A$, reading the weight matrix $W$, and writing the output $C$:

$$\begin{aligned}\text{Bytes}_{\text{GEMM}} &= e_b \times \big(BND + D^2 + BND\big) \\ &= e_b(2BND + D^2)\end{aligned} \tag{7}$$

The operational intensity is therefore:

$$\text{OI}_{\text{GEMM}} = \frac{2BND^2}{e_b(2BND + D^2)} \tag{8}$$

At large $D$ (where $D^2 \gg 2BND$ for small $B, N$), this simplifies to $\text{OI}_{\text{GEMM}} \approx D/(e_b)$, which grows linearly with $D$. This is the key property that allows GEMM to transition from memory-bound to compute-bound as problem size increases – the weight matrix $W$ is reused across all $BN$ input rows, amortizing its loading cost.

### 5.2 Element-wise Add

The residual connection $C = A + B_{\text{res}}$ performs one floating-point addition per element:

$$\text{FLOPs}_{\text{Add}} = B \times N \times D \tag{9}$$

Memory traffic requires reading both inputs and writing the output:

$$\text{Bytes}_{\text{Add}} = e_b \times 3 \times B \times N \times D \tag{10}$$

$$\text{OI}_{\text{Add}} = \frac{1}{3e_b} = \begin{cases} 0.083 & \text{FP32} \\ 0.167 & \text{FP16} \end{cases} \tag{11}$$

This OI is constant, independent of $B$, $N$, or $D$. It is far below both ridge points, making element-wise Add persistently memory-bound regardless of problem size or precision.

### 5.3 Softmax

The standard two-pass softmax over the attention matrix $A \in \mathbb{R}^{B\times N\times N}$ performs: (i) a max subtraction for numerical stability, (ii) an exponentiation, (iii) a summation, and (iv) a division, approximately $5N$ FLOPs per row:

$$\text{FLOPs}_{\text{Softmax}} = B \times N \times 5N = 5BN^2 \tag{12}$$

The two-pass algorithm reads the attention matrix once for the max and once for the exp/sum pass, then writes the normalized output:

$$\text{Bytes}_{\text{Softmax}} = e_b \times 2 \times B \times N^2 \tag{13}$$

$$\text{OI}_{\text{Softmax}} = \frac{5BN^2}{2e_b \cdot BN^2} = \frac{5}{2e_b} = \begin{cases} 0.625 & \text{FP32} \\ 1.250 & \text{FP16} \end{cases} \tag{14}$$

Like element-wise Add, this OI is size-invariant, Softmax always remains memory-bound. The quadratic scaling of both FLOPs and bytes with $N$ means that the total memory traffic grows as $O(N^2)$, motivating attention-focused bandwidth optimizations such as FlashAttention (2).

## 6 Benchmarking Harness

### 6.1 Measurement Protocol

Accurate GPU latency measurement requires careful handling of CUDA asynchrony. Our harness addresses all three pitfalls identified in Section 2:

**Warm-up.** Each kernel receives 20 warm-up iterations before timing begins. This discards JIT compilation time, cuBLAS plan selection, and CUDA context initialization. Without warm-up, the first few iterations can be 10–100× slower than steady-state.

**Synchronization.**

We call `torch.cuda.synchronize()` before the timed window (to flush any prior async work) and after the timed window (to ensure the GPU has completed all launched work before the clock stops). Using wall-clock time (`time.perf_counter`) with synchronization is equivalent to using CUDA events and avoids any CPU dispatch artifact.

**Averaging.** We run 100 iterations inside the timed window and divide the total elapsed time by 100. This averages out micro-jitter from OS scheduling and cache effects. We repeat this three times independently and report mean ± standard deviation in milliseconds.

**Reproducibility.**

We set `torch.manual_seed(42)`, `torch.backends.cudnn.deterministic=True`, and `torch.backends.cudnn.benchmark=False` to ensure consistent kernel selection and timing across runs.

### 6.2 Sweep Configuration

We sweep five problem sizes $D = N \in \{256, 512, 1024, 2048, 4096\}$ with batch size $B =$

1 (representative of inference), and two precisions: FP32 ($e_b = 4$) and FP16 ($e_b = 2$). This yields $5 \times 2 \times 3 = 30$ distinct experiment configurations. Achieved GFLOPs for each configuration is computed as:

$$\text{GFLOPs}_{\text{achieved}} = \frac{\text{FLOPs}}{t_{\text{ms}}/1000 \times 10^9} \tag{15}$$

# 7 Experimental Results

## 7.1 Latency Scaling

Figure 2 plots per-kernel latency as a function of $D = N$ for both precisions. GEMM latency grows rapidly with $D^2$ (as expected from its $O(D^3)$ FLOPs), while FP16 consistently achieves lower latency due to Tensor Core acceleration and halved memory traffic. Element-wise Add and Softmax grow more slowly, and the gap between FP32 and FP16 is modest, a sign that these kernels are not compute-limited.

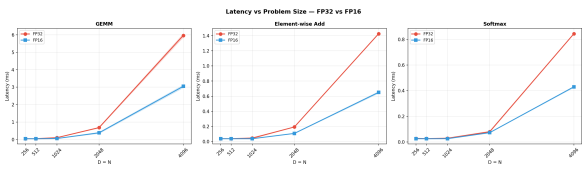


Figure 2: Latency vs. problem size ($D = N$) for GEMM, element-wise Add, and Softmax. Shaded bands represent $\pm 1$ standard deviation across three independent repeats. FP16 offers a large latency reduction for GEMM but only modest improvement for Add and Softmax, consistent with their memory-bound classification.

## 7.2 Operational Intensity vs. Problem Size

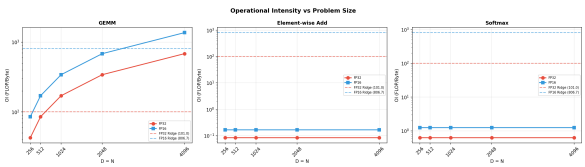


Figure 3: Operational intensity vs. problem size for all three kernels. Horizontal dashed lines show FP32 (red) and FP16 (blue) ridge points. GEMM OI grows linearly with $D$, crossing the FP32 ridge at large sizes. Add and Softmax OI are size-invariant.

Figure 3 plots measured OI against $D$ for each kernel. The dashed horizontal lines mark the FP32 and FP16 ridge points. GEMM OI grows linearly with $D$, confirming the analytical prediction from Equation (3). For $D = 4096$, the FP32 GEMM OI reaches approximately 682 FLOP/Byte, well above the FP32 ridge of 101, confirming compute-bound behavior at scale. Element-wise Add OI remains flat at 0.083 (FP32) and 0.167 (FP16), orders of magnitude below both ridge points. Softmax OI is similarly flat at 0.625 (FP32) and 1.250 (FP16).

## 7.3 Kernel Roofline Map

Figure 4 overlays all kernel measurements on the Roofline chart. Each point represents one kernel at one problem size. The trajectory of GEMM points moves along a diagonal from the lower-left (memory-bound, small sizes) toward the compute ceiling (large sizes). Element-wise Add and Softmax cluster near the left side of the chart, hugging the memory-bandwidth slope at all measured sizes. The FP16 roofline (right panel) has a $8\times$ higher compute ceiling, yet Add and Softmax remain far from it, only GEMM at the largest sizes begins approaching compute saturation.

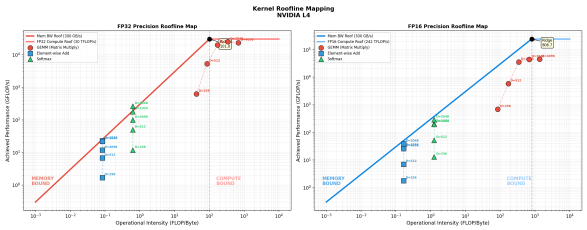


Figure 4: Main result: all kernels mapped onto the FP32 (left) and FP16 (right) rooflines. GEMM transitions from memory-bound to compute-bound with increasing $D$. Element-wise Add and Softmax remain memory-bound at all sizes.

## 7.4 FP32 vs. FP16 Comparison

Figure 5 compares achieved GFLOPs (top row) and OI (bottom row) for FP32 and FP16 across all kernel sizes. FP16 doubles the OI for Add and Softmax (halved $e_b$), but since their OI remains far below both ridge points, this does not translate into a meaningful throughput gain, the kernel is still bandwidth-limited. For GEMM, FP16 provides both doubled OI and a $8\times$ higher compute ceiling, delivering a compound throughput advantage that grows with $D$.

## 7.5 Scaling Classification

Figure 6 visualizes how the bound classification of each kernel shifts with problem size. Gold-bordered points are compute-bound; black-bordered points are memory-bound. GEMM (red circles) transitions from black to gold at large $D$ in FP32, while Add and Softmax remain black at

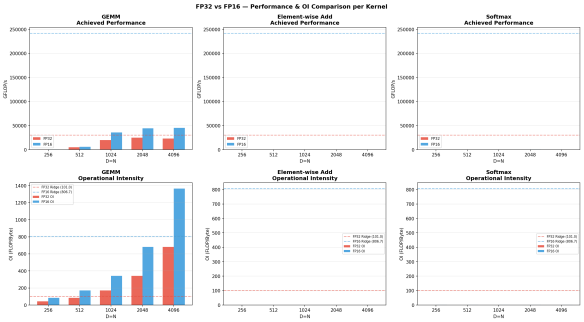


Figure 5: FP32 vs. FP16 comparison: achieved GFLOPs (top) and operational intensity (bottom) for all three kernels across problem sizes.

every size. Table 1 formalizes this classification with OI and GFLOPs ranges.

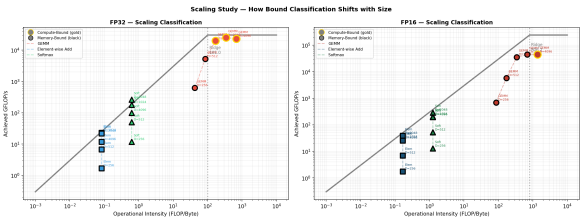


Figure 6: Scaling study: gold borders indicate compute-bound classification; black borders indicate memory-bound. GEMM transitions with size; Add and Softmax do not.

| Kernel | Prec | OI Range | Classification |
|---|---|---|---|
| GEMM | FP32 | 42.7–682.7 | Shifts M→C |
| GEMM | FP16 | 85.3–1365.3 | Shifts M→C |
| Add | FP32 | 0.083 (const) | Always Mem.-Bound |
| Add | FP16 | 0.167 (const) | Always Mem.-Bound |
| Softmax | FP32 | 0.625 (const) | Always Mem.-Bound |
| Softmax | FP16 | 1.250 (const) | Always Mem.-Bound |

Table 1: Bound classification summary for all kernel/precision combinations.

### 7.6 System-Level Profiling

To complement the roofline analysis, we replicate a Nsight Systems timeline using CUDA event timers and NVTX-annotated profiling scripts. This provides per-launch kernel duration and CPU dispatch time at microsecond resolution.

**Execution timeline.** Figure 7 shows the reconstructed GPU execution timeline for all three kernels across 8 repetitions at $D = N = 512$. GEMM dominates wall time in both precisions. Add and Softmax kernels are visible as brief segments between GEMM launches.

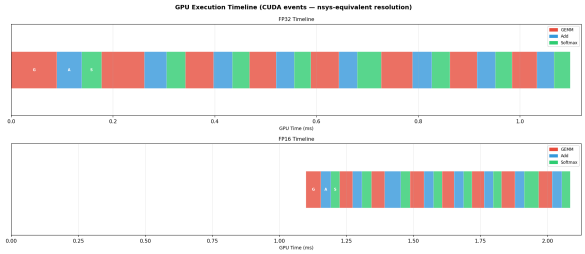


Figure 7: Reconstructed GPU execution timeline (CUDA events, nsys-equivalent resolution) for FP32 (top) and FP16 (bottom) at $D = N = 512$. GEMM (red) dominates wall time.

**CPU dispatch vs. GPU compute.** Figure 8 compares CPU-side dispatch latency against GPU execution time for each kernel. For small problem sizes (D=N=512), CPU dispatch overhead is comparable to GPU compute time for Add and Softmax, indicating that for these lightweight kernels, the CPU is not keeping the GPU fully fed. At larger problem sizes this ratio inverts: GPU compute time dominates and the GPU becomes the bottleneck. This insight motivates kernel fusion strategies that reduce launch overhead by combining multiple lightweight operations into a single GPU kernel.

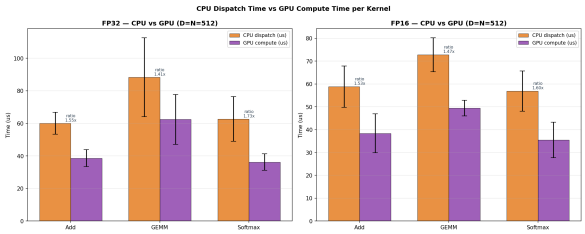


Figure 8: CPU dispatch time vs. GPU compute time per kernel at $D = N = 512$. For Add and Softmax, CPU overhead is non-negligible, motivating kernel fusion.

**Duration sweep.** Figure 9 shows per-kernel GPU duration versus problem size on a log scale. GEMM duration grows as $O(D^3)$, while Softmax grows as $O(N^2)$ due to the quadratic attention matrix. This confirms the HBM-bound $O(N^2)$ scaling of Softmax and is the key motivation for FlashAttention, which avoids materializing the full $N \times N$ attention matrix in HBM.

## 8 Optimization Strategy

The Roofline classification directly dictates the correct optimization strategy for each kernel. The core principle is: *optimization is only meaningful if it targets the actual bottleneck.*

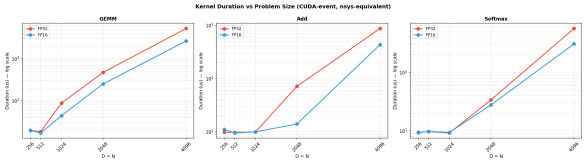


Figure 9: Kernel duration vs. problem size on a log scale (CUDA event timing, nsys-equivalent). GEMM grows as $O(D^3)$; Softmax grows as $O(N^2)$.

**GEMM: Tensor Cores and Tiling.** At large hidden dimensions, GEMM is compute-bound. The correct optimization is to maximize FLOPs per cycle: use FP16/BF16 to activate Tensor Cores, which provide a nominal $8\times$ compute speedup on L4. At small sizes where GEMM remains memory-bound, INT8 or INT4 quantization is more effective, it reduces $e_b$ and thus increases OI without needing more FLOP throughput.

**Element-wise Add: Kernel Fusion.** With a constant OI of 0.083 (FP32), Add cannot benefit from Tensor Cores. The only effective optimizations reduce byte traffic: (1) kernel fusion with adjacent operations (e.g., fuse Add with layer normalization to eliminate intermediate HBM writes), (2) in-place operations to remove the output write allocation, and (3) contiguous memory layout to ensure coalesced HBM access patterns.

**Softmax: FlashAttention.** Softmax is limited by the two HBM round-trips of its two-pass algorithm and by the $O(N^2)$ scaling of the attention matrix. FlashAttention (2) resolves both by implementing a fused, tiled kernel that keeps intermediate values in SRAM and never materializes the full attention matrix in HBM. This reduces HBM traffic from $O(N^2)$ to $O(N)$ per head, directly addressing the bandwidth bottleneck.

## 9 Conclusion

We have presented a complete benchmarking and profiling study for three LLM kernels on a modern GPU. By constructing a Roofline Model, deriving analytical FLOPs and memory formulas, and measuring empirical performance across problem sizes and precisions, we show that LLM kernels are heterogeneously bounded: GEMM transitions from memory-bound to compute-bound with scale, while element-wise Add and Softmax are persistently memory-bound. This heterogeneity means a single optimization strategy is insufficient; GEMM benefits from Tensor Cores, while Add and Softmax benefit from kernel fusion and bandwidth reduction. The system-level timeline analysis further shows that CPU dispatch overhead is non-trivial for lightweight memory-bound kernels, reinforcing the value of fusion. We hope this work serves as a reproducible template for rigorous performance analysis in deep learning systems.

## Author Contributions

**Subhon Ghosh** built the Roofline Model, measuring empirical GPU peak bandwidth and compute, deriving the theoretical and attainable ceilings, and producing the dual-precision roofline plots.

**Cheng-Lun Tai** (Allen) handled kernel mapping: deriving the operational intensity values and achieved performance for all three kernels across sizes and precisions, and integrating those measurement points onto the Roofline chart.

**Shahid Mulla** performed the kernel profiling and deeper analysis: benchmarking GEMM, element-wise Add, and Softmax across sizes 256–4096 in both FP32 and FP16, integrating the profiling results with the Roofline model, exploring Nsight Systems and related tooling, and producing the final kernel-on-roofline summary. All three authors contributed equally to writing the report and preparing the presentation.

## References

[1] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM, 52(4):65–76, 2009.

[2] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In Advances in Neural Information Processing Systems (NeurIPS), 2022.

[3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Advances in Neural Information Processing Systems (NeurIPS), 2017.

[4] NVIDIA Corporation. Nsight Systems user guide. `https://docs.nvidia.com/nsight-systems/`, 2024.

[5] PyTorch Contributors. PyTorch profiler documentation. `https://pytorch.org/docs/stable/profiler.html`, 2024.

[6] Aleksandar Ilić, Frederico Pratas, and Leonel Sousa. Cache-aware Roofline model: Upgrading the loft. IEEE Computer Architecture Letters, 13(1):21–24, 2014.

[7] Google LLC. Perfetto: Production-grade system tracing. `https://perfetto.dev/`, 2024.

[8] NVIDIA Corporation. NVIDIA L4 Tensor Core GPU datasheet. `https://www.nvidia.com/en-us/data-center/l4/`, 2023.